Hierarchical Rotary Position Embedding (HRoPE)

> Incremental inference for LLMs via sentence-level caching, re-rotatable KV, and ReLU-native attention

Feng Zhou — April 2026 `Patent Pending` `Technical Specification v3.0`

---

1. Problem Statement

Standard LLMs recompute the entire KV cache when any token changes. For a 100k-token document, editing one word triggers $O(N^2)$ recomputation — several seconds of GPU time. This makes iterative editing prohibitively expensive and prevents true **incremental compilation** for LLMs.

Our Goal: Edit a sentence, recompute only that sentence. Everything else is served from a precomputed, position-agnostic cache.

> 💡 **Analogy — nnU-Net for Medical Imaging:** nnU-Net processes images at multiple resolutions (organ → tissue → cell). HRoPE processes text at multiple levels (paragraph → sentence → token). This hierarchical structure enables efficient, incremental updates and multi-scale pattern recognition.

2. Core Mathematical Insight: The Composability of Rotation

RoPE (Rotary Position Embedding) applies a rotation to key vectors based on absolute position. For a vector $x \in \mathbb{R}^d$ with d even:

$$\mathrm{RoPE}(x, p) = R(p\theta) \cdot x$$

> p: absolute position
> $\theta_i = 10000^{-2i/d}$ for $i = 0, 1, \ldots, d/2 - 1$
> $R(p\theta)$: block-diagonal rotation matrix

Rotation matrices are **composable** ($R(\theta_1)R(\theta_2) = R(\theta_1 + \theta_2)$) and **invertible** ($R(\theta)^{-1} = R(-\theta)$).

If we store the **unrotated** key vectors K_{base}, we can rotate them to any target position later:

$$K_{\text{cached}} = R(p_{\text{old}}\theta) \cdot K_{\text{base}}$$

$$K_{\text{new}} = R(p_{\text{new}}\theta) \cdot K_{\text{base}} = R((p_{\text{new}} - p_{\text{old}})\theta) \cdot K_{\text{cached}}$$

This means a sentence's KV cache is **position-neutral**. It works anywhere in any document. No approximation — the math is exact.

3. Two Implementation Paths

🔧 Path 1: Retrofit with ReLU/Norm Fine-Tuning

Our approach, distinct from KVLINK. Instead of adding link tokens, we replace softmax with ReLU + renormalization and fine-tune the model. This eliminates the softmax denominator problem at its source. Optionally, we use a small **correction model** to recover LayerNorm leakage. Target: exceed KVLINK's accuracy (2-5% drop) while maintaining 90%+ speedup.

🚀 Path 2: Native HRoPE Training

Train new models from scratch with hierarchical position awareness and ReLU-native attention. Enables true O(Δ) inference and 10-20× context extrapolation. Ultimate goal: match or exceed standard softmax accuracy with 100x+ faster editing.

4. Path 1 Deep Dive: ReLU/Norm Fine-Tuning

KVLINK uses trainable **link tokens** to restore cross-document attention. We take a different approach: modify the attention mechanism itself.

4.1 Why Replace Softmax with ReLU?

Standard softmax causes the denominator to change when stitching caches. ReLU attention eliminates this:

$$\text{Attention}_{\text{ReLU}}(Q, K, V) = \frac{\text{ReLU}(QK^T)}{\sum \text{ReLU}(QK^T)} V$$

Properties:

- ReLU is piecewise linear — enables delta updates
- No denominator normalization across the full sequence — stitching becomes additive
- Empirically achieves comparable quality to softmax with proper initialization

4.2 The LayerNorm Leakage Problem & Our Correction Model

When stitching independently encoded sentences, LayerNorm statistics drift. We introduce a **small correction model** (2-4 layers, ~50M parameters) that runs after cache retrieval:

$$\text{Corrected}(K) = K + \text{MLP}(K, \text{context}\backslash_\text{stats})$$

The correction model is trained to predict the residual between stitched and full-encoding outputs. This recovers the "leakage" without full recomputation.

4.3 Fine-Tuning Protocol

Component	Setting
Base model	Llama-3.2-3B or Qwen-1.5B

Component	Setting
Attention modification	Replace softmax with ReLU + renormalization
Training data	Mixed: QA with shuffled documents + summarization + multi-turn
Hardware	8× A100 GPUs
Steps	6,000 (similar to KVLINK)
Correction model size	~50M parameters (optional)

4.4 Expected Performance vs. KVLINK

Metric	KVLINK (link tokens)	HRoPE Path 1 (ReLU + correction)
Accuracy vs. upper bound	~2-5% drop	**~1-3% drop (target)**
Inference speedup	85-96% TTFT reduction	**90-96% TTFT reduction**
Additional parameters	Link tokens (few hundred)	Correction model (~50M)
Fine-tuning cost	6k steps on 8×A100	6k steps + correction training
Delta computation support	No (still softmax)	**Yes (ReLU is linearizable)**

5. Mathematical Formulation: Two-Level Hierarchy

For a token at sentence index i and token offset j:

$$\Theta(p) = \alpha_{\text{sent}} \cdot i + \alpha_{\text{token}} \cdot j$$

> i: sentence index (0, 1, 2, ...)
> j: token offset within sentence (0, 1, ..., $L_i - 1$)

> α_{sent}: sentence-level frequency (slow rotation)
> α_{token}: token-level frequency (fast rotation)

The attention score between tokens at (i, j) and (i', j') depends on:

$$\Delta\Theta = \alpha_{\text{sent}}(i - i') + \alpha_{\text{token}}(j - j')$$

5.1 Variable-Length Sentences (No Fixed Limits)

$$\text{Document} = [S_1, S_2, \ldots, S_N], \quad S_i = [t_{i,1}, \ldots, t_{i,L_i}]$$

> N: number of sentences (variable)
> L_i: number of tokens in sentence i (variable per sentence)

5.2 ALiBi for Cross-Segment Bias (Global Order)

To handle the global order of segments, we integrate **ALiBi (Attention with Linear Biases)** based on segment index difference:

$$\text{Attention}_{\text{Global}}(Q, K) = QK^T - m \cdot |\text{seg}_i - \text{seg}_j|$$

> 📚 **Literature Reference:** The 2025 **BiPE (Bilevel Positional Encoding)** paper validates this approach, achieving 97% accuracy on arithmetic tasks vs. <70% baseline.

6. The Re-Rotation Kernel (Triton)

```
@triton.jit
def rerotate_kv_kernel(K_base_ptr, K_out_ptr, start_pos, seq_len, d_head):
    token_idx = tl.program_id(0)
    dim_idx = tl.program_id(1)

    theta = 10000.0 ** (-2.0 * dim_idx / d_head)
```

```
    angle = start_pos * theta

    cos_theta = tl.cos(angle)
    sin_theta = tl.sin(angle)

    x = tl.load(K_base_ptr + token_idx * d_head + dim_idx*2)
    y = tl.load(K_base_ptr + token_idx * d_head + dim_idx*2 + 1)

    tl.store(K_out_ptr + token_idx * d_head + dim_idx*2,
             x * cos_theta - y * sin_theta)
    tl.store(K_out_ptr + token_idx * d_head + dim_idx*2 + 1,
             x * sin_theta + y * cos_theta)
```

7. LSH-Based Fuzzy Cache (SimHash)

```
def simhash(sentence, hash_bits=64):
    features = sentence.lower().split()
    v = [0] * hash_bits

    for f in features:
        h = hashlib.md5(f.encode()).digest()
        h_int = int.from_bytes(h, 'big')
        for i in range(hash_bits):
            if (h_int >> i) & 1:
                v[i] += 1
            else:
                v[i] -= 1

    signature = 0
    for i in range(hash_bits):
        if v[i] > 0:
            signature |= (1 << i)
    return signature

# "The cat sat" → 0x7f3a2b1c...
# "The dog sat" → 0x7f3a2b1d... (Hamming distance = 1-2 bits)
```

8. Performance Projections

Scenario	Standard	KVLINK	HRoPE Path 1	Native HRoPE
100k tokens, edit 1 sentence	~5 sec	~0.5-1 sec	**~0.4-0.8 sec**	~0.1-0.3 sec
Accuracy vs. upper bound	N/A	~2-5% drop	**~1-3% drop**	~0% (target)
Cache hit rate	0%	85-95%	**85-95%**	90-98%
Context extrapolation	2-4×	2-4×	2-4×	**10-20×**

9. Native HRoPE Training (Path 2)

For next-generation models, we train from scratch with:

- **Hierarchical RoPE:** Separate learned frequencies for sentence and token levels
- **ReLU-native attention:** No softmax, enabling true O(Δ) inference
- **ALiBi for cross-segment:** Built-in length extrapolation

9.1 Training Stability for ReLU Attention

- Embedding standard deviation = 1
- Attention/FFN layer initialization $\sigma \approx 0.02$
- Gradually increase sequence length during training (512 → 2K → 8K → 32K+)

9.2 Compute Requirements

Model Size	GPUs	Time	Cost (Cloud)
1.5B (proof)	32×A100	18 days	~$50K
7B (production)	128×H100	30 days	~$200K

10. Literature Review & Differentiation from KVLINK

> 🎯 **How we exceed KVLINK:** KVLINK adds link tokens but keeps softmax. We replace softmax with ReLU, eliminating the denominator problem at its

> source. Our correction model recovers LayerNorm leakage. This enables true delta computation (O(Δ)) and better accuracy.

- **KVLINK (2025):** Link tokens + positional re-encoding. Still uses softmax. Accuracy drop ~2-5%.
- **ReLU Attention (2021-2025):** Comparable quality to softmax with linear complexity. Enables additive updates.
- **BiPE (2025):** Validates bilevel positional encoding (local RoPE + global ALiBi).
- **Our Contribution:** First to combine ReLU attention + hierarchical RoPE + correction model for stitchable KV caches.

11. Current Status & Roadmap

Completed: Re-rotation kernel, SimHash cache, delta recompute logic, Llama integration.

In Progress: ReLU fine-tuning pipeline, correction model training.

Next Steps:

- Benchmark ReLU fine-tuning vs. KVLINK on LongBench
- Train correction model for LayerNorm leakage
- Native HRoPE training at 1.5B scale

12. Provisional Patent Claims (Updated)

- **Claim 1:** ReLU-based linearized attention for stitchable KV caches in LLM inference
- **Claim 2:** Correction model for LayerNorm leakage recovery during cache stitching
- **Claim 3:** Zero-angle KV storage with re-rotation by position difference
- **Claim 4:** Hierarchical RoPE with separate sentence and token frequencies
- **Claim 5:** Bilevel positional encoding (local RoPE + global ALiBi) for cross-segment attention

> 📫 **Contact:** Feng Zhou — Technical portfolio for L7 interview at Google. This document is part of an ongoing research project aiming to exceed KVLINK's accuracy.

© 2026 Feng Zhou — HRoPE: Hierarchical Rotary Position Embedding. Confidential / Patent Pending.